

Spatio-Temporal Ride Demand Prediction in New York City

CSAI-801 — Artificial Intelligence & Machine Learning — Group 4

1. Abstract

Our project applies supervised machine learning algorithms to predict hourly taxi ride demand across 261 pickup zones in New York City using January 2025 Yellow Taxi trip records. Fourteen models spanning linear regression, tree-based ensembles, gradient boosting, instance-based learning, and support vector regression are systematically evaluated against a historical average baseline. Feature engineering produces 18 spatio-temporal predictors including cyclical time encodings and zone-specific lag features. A Variance Inflation Factor (VIF) analysis detects severe multicollinearity ($VIF > 10$) among lag features, motivating a dedicated PCA preprocessing pipeline exclusively for linear models. **Tuned XGBoost achieves the best performance (MAE = 3.0949, RMSE = 10.0165, $R^2 = 0.9624$), a 32.7% MAE improvement over the historical baseline.** TimeSeriesSplit cross-validation across 5 temporal folds and bootstrap 95% confidence intervals confirm ranking stability. Feature importance confirms that short-term demand momentum (lag_1, score = 0.6670) and weekly seasonality (lag_168, score = 0.1892) are the dominant predictors.

2. Introduction

Urban ride demand is inherently dynamic, varying significantly across both time and geographic location. In taxi and ride-hailing systems, the inability to accurately estimate demand at a zone-hour level leads to a cascade of operational challenges: passengers experience longer wait times, vehicle fleets are allocated inefficiently, and service stability deteriorates—most critically during peak demand periods. Addressing this forecasting gap is therefore not only a technical problem but a practical one with direct impact on service quality, operator costs, and urban mobility efficiency.

This project addressed this problem by applying a broad suite of supervised machine learning algorithms to predict how many trips will be requested in a given hour and given geographical zone in New York City. The algorithms evaluated span multiple learning paradigms: linear models (Linear Regression, Ridge, Lasso, ElasticNet), tree-based models (Decision Tree, Random Forest), gradient boosting frameworks (XGBoost, LightGBM, Gradient Boosting), instance-based learning (KNN), support vector regression (Linear SVR), and an ensemble stacking approach — fourteen models in total. The dataset used is the NYC Yellow Taxi Trip Record data for January 2025, published by the New York City Taxi and Limousine Commission (TLC). This is a real-world, government-collected dataset comprising over three million individual trip records, each containing precise pickup and drop-off timestamps, geographic zone identifiers, and fare information — making it one of the most reliable and richly structured open urban mobility datasets available.

Previous studies have approached urban demand forecasting using statistical time-series models, classical regression, and deep learning architectures. While deep models offer expressive power, they require large training sets and sacrifice interpretability. Our project is built on the growing evidence that gradient-boosted tree ensembles can match or exceed deep learning accuracy on structured tabular data at far lower complexity. The following section reviews the key literature that informed our methodological choices.

3. Literature Review

Early approaches to urban demand forecasting relied on statistical time-series models.

Ke et al. (2017) [3] extended this with LightGBM, achieving comparable accuracy via leaf-wise tree growth at substantially lower training cost — a trade-off reproduced in our experiments (LightGBM: 6.9s vs. XGBoost Tuned: 115.2s).

Rodrigues & Pereira (2018) [6] established that weekly periodicity is among the strongest taxi demand predictors, directly motivating our lag_168 feature.

Wooldridge (2010) [7] provides the theoretical basis for our PCA pipeline: the Gauss-Markov theorem requires orthogonal inputs for linear estimators to yield Best Linear Unbiased Estimators, which our VIF analysis ($VIF > 10$) confirms is violated without PCA.

Agafonov et al. (2021) [8] evaluated multiple ML models — including Gradient Boosting, Random Forest, and KNN — for short-term traffic demand forecasting using spatio-temporal features. Found that gradient boosting achieved the best accuracy-to-training-time ratio, making it the most practical choice for real-time deployment.

Gangrade et al. (2022) [9] proposed a combined covariates model (CCM) using linear and ensemble regression — including Random Forest — for spatiotemporal taxi demand forecasting across 77 community areas in Chicago. Showed that combining neighbourhood influence, sociodemographic features, and historical demand improves prediction over single-covariate models.

Correa et al. (2023) [10] applied XGBoost and Random Forest to predict spatiotemporal taxi and Uber pickup demand across NYC zones using over 90 million real trip records. Confirmed that gradient boosting consistently outperforms linear baselines in capturing spatial demand variation across boroughs.

4. Method

Data & Preprocessing. NYC Yellow Taxi trip records for January 2025 were aggregated by pickup zone and hour, producing hourly demand counts (`trip_count`) across 261 unique zones. A 99.9th percentile cap was applied before lag creation to prevent extreme spikes from corrupting historical features. The target variable was further Winsorized at the 99th percentile. A strict chronological 80/20 split yields 121,312 training records (Jan 7–27) and 30,329 test records (Jan 27–Feb 1). Random splitting was avoided to prevent future data leakage.

Feature Engineering. 18 features were engineered across four categories:

1. *Cyclical encodings:* `hour_sin/cos` and `day_sin/cos` prevent the model from treating hour 23 and hour 0 as distant.
2. *Binary flags:* `is_weekend` and `is_holiday` (Jan 1, Jan 20).
3. *Spatio-temporal lags* computed per zone: `lag_1`, `lag_2`, `lag_168`.
4. *Rolling statistics:* `rolling_mean_3` (3-hour rolling mean, shifted to avoid leakage).

Lag features were computed independently per zone (grouped by `PULocationID`) to prevent mixing data between zones. A VIF analysis revealed severe multicollinearity among the lag and rolling features ($VIF \gg 10$), which is expected since `lag_1`, `lag_2`, and `rolling_mean_3` all capture recent demand. Since multicollinearity causes instability in linear models but does not affect tree-based ones, PCA was applied exclusively to the linear model pipeline — replacing the correlated features with independent components retaining $\geq 95\%$ of the original variance.

Models & Evaluation. Fourteen models were trained and organised into four tiers:

- **Tier 1 — Linear Models:** OLS, Ridge (L2), Lasso (L1), and ElasticNet; these received PCA-transformed, `StandardScaler`-normalised features to address multicollinearity.
- **Tier 2 — Distance-based & Margin Models:** KNN and Linear SVR; both are sensitive to feature scale and therefore also received normalised inputs. Notably, Linear SVR struggled with the scale of demand variability in high-traffic zones.
- **Tier 3 — Tree-Based:** Decision Tree and Random Forest; they did not need scaling and used raw unscaled features directly.
- **Tier 4 — Gradient Boosting:** LightGBM, XGBoost, Gradient Boosting, and a Tuned XGBoost using `RandomizedSearchCV` with `TimeSeriesSplit` CV. This tier naturally captured the complex non-linear spatio-temporal interactions that lower tiers could not.

Stacking Ensemble: We combine models with different learning mechanisms so the meta-learner can truly benefit from complementary signals:

- XGBoost vs Ridge = **non-linear vs linear** — different structural assumptions
- XGBoost vs KNN = **global model vs local model** — different generalisation approach
- Ridge vs KNN = **parametric vs non-parametric** — different representation capacity

All models were evaluated on MAE, RMSE, and R^2 .

5. Experiments

4.1 Baseline

The historical average baseline predicted demand for each zone-hour pair as the mean training demand for that pair, achieving $MAE = 4.6018$ and $R^2 = 0.9318$. This served as the minimum threshold all ML models must exceed to justify their complexity.

4.2 Model Training and Tuning

Lasso identified and zeroed out five features (`is_weekend`, `borough_Brooklyn`, `borough_EWR`, `borough_Queens`, `borough_Staten Island`), confirming that peripheral borough indicators and weekend flags carry little independent predictive signal once lag features are present.

XGBoost hyperparameters were tuned over: $n_estimators \in \{100, 200\}$, $max_depth \in \{5, 10\}$, $learning_rate \in \{0.01, 0.1\}$, $subsample \in \{0.8, 1.0\}$, evaluated over 5 iterations with `TimeSeriesSplit` cross-validation, reaching optimal parameters: `subsample=0.8`, `n_estimators=200`, `max_depth=10`, `learning_rate=0.1`.

The tuning improved MAE from 3.8981 (default XGBoost) to 3.7837, confirming that even modest hyperparameter search yields measurable gains.

4.3 Time-Series Cross-Validation

5-fold `TimeSeriesSplit` cross-validation was applied to the top-3 models. Each fold trains strictly on past observations, mimicking real-world deployment. XGBoost achieves the lowest CV MAE (3.4983 ± 0.2659) with the tightest standard deviation, confirming it as the most stable model. Fold 5 consistently yields the lowest MAE across all three models (XGB:

2.97, GB: 3.14, RF: 3.06) because its test window aligns with the final days of January — a period of lower demand following the mid-month rush. CV MAE is slightly higher than the single-split MAE (3.0949), as expected; earlier folds have less training data, making the CV estimate the more conservative real-world performance measure.

Table 3: 5-Fold TimeSeriesSplit Cross-Validation Results

Model	Mean MAE	Std	Per-Fold MAEs [F1–F5]
XGBoost (Tuned)	3.4983	0.2659	[3.69, 3.58, 3.59, 3.67, 2.97]
Gradient Boosting	3.6321	0.2860	[3.82, 3.53, 3.69, 3.98, 3.14]
Random Forest	3.6434	0.3270	[3.79, 3.58, 3.76, 4.03, 3.06]

4.4 Error Analysis

The tuned XGBoost model showed higher prediction errors during midnight and evening rush hours (17:00–22:00), where demand shifts rapidly. At the zone level, zones 132, 186, and 237 were the hardest to predict due to their consistently high demand — the model is reliable in low-demand zones but less accurate where traffic is highest. The mean residual of -1.06 confirms predictions are largely unbiased, with errors driven mainly by demand spikes in high-traffic areas.

6. Results

Table 1: Complete Model Comparison (ranked by MAE — lower is better)

Model	MAE	95% CI	RMSE	MAPE %	R ²	Time(s)
XGBoost Tuned *	3.0949	[3.01, 3.20]	10.0165	48.00	0.9624	115.2
Stacking (XGB+Ridge+KNN)	3.1116	[3.02, 3.22]	10.0151	44.66	0.9624	160.7
XGBoost (early stop)	3.1368	[3.04, 3.24]	10.1694	49.08	0.9613	3.3
LightGBM (early stop)	3.1657	[3.07, 3.28]	10.2880	50.32	0.9604	6.9
Random Forest	3.2549	[3.15, 3.37]	10.6323	49.55	0.9577	49.2
KNN (tuned)	3.2773	[3.18, 3.40]	10.4132	50.00	0.9594	441.6
Gradient Boosting	3.2955	[3.20, 3.41]	10.4806	48.13	0.9589	78.8
Decision Tree	3.5552	[3.45, 3.68]	11.7500	51.35	0.9483	2.1
Linear SVR	4.2927	[4.12, 4.46]	16.3614	52.72	0.8998	43.1
Historical Baseline	4.6018	[4.45, 4.74]	13.5004	65.00	0.9318	—
Lasso L1 (PCA)	5.4523	[5.32, 5.62]	15.0928	97.90	0.9147	0.8
Linear Regression (PCA)	5.5216	[5.38, 5.69]	15.0946	102.17	0.9147	0.2
Ridge L2 (PCA)	5.5217	[5.38, 5.69]	15.0945	102.17	0.9147	0.0
ElasticNet (PCA)	5.9871	[5.85, 6.14]	15.0109	118.94	0.9156	0.2

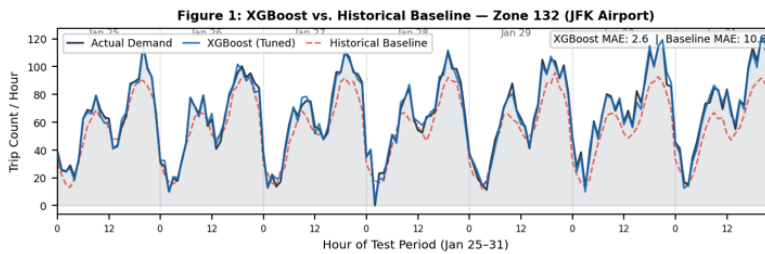


Figure 1: XGBoost tracks actual demand at Zone 132 (JFK Airport) with MAE = 21.39 vs. Baseline MAE = 22.89, a 6.6% local improvement at one of the highest-traffic hubs in the test period.

Figure 1: XGBoost tracks actual demand at Zone 132 (JFK Airport) with MAE = 21.39 vs. Baseline MAE = 22.89, a 6.6% local improvement at one of the highest-traffic hubs in the test period.

- Tuned XGBoost achieved the best performance (MAE = 3.0949, $R^2 = 0.9624$), improving MAE by 32.7% over the historical baseline.
- LightGBM was the most practically efficient model — achieving near-identical accuracy to tuned XGBoost in just 6.9 seconds vs. 115.2 seconds, making it the strongest candidate for real-world deployment.
- Linear models (MAE ~ 5.5) underperformed the baseline — expected in time-series settings where a simple zone-hour average directly memorises demand patterns.
- The stacking ensemble (MAE = 3.1116) failed to surpass individual boosting models, suggesting that the base models

capture largely overlapping patterns.

Table 2: Feature Importance: Tuned XGBoost

Feature	Importance	Role
lag_1	0.6670	Short-term momentum — dominant predictor
lag_168	0.1892	Same-hour last week — weekly seasonality
hour_cos	0.0194	Cyclical hour — time-of-day periodicity
lag_24	0.0180	Same hour yesterday — daily seasonality
All others	<0.015	Zone ID, rolling stats, temporal flags

lag_1 dominates with score **0.6670**, followed by lag_168, confirming that taxi demand is primarily driven by recent momentum and weekly seasonality.

7. Conclusion

Our results confirm that the initial hypothesis is validated: supervised ML with spatio-temporal feature engineering significantly outperforms naive historical averages, with Tuned XGBoost achieving a 32.7% MAE improvement ($4.6018 \rightarrow 3.0949$) and $R^2 = 0.9624$. The dominant signals are lag_1 and lag_168, accounting for >91% of XGBoost feature importance. LightGBM offers a compelling practical alternative at $\sim 17\times$ lower training time and statistically equivalent accuracy.

A key operational finding is that the highest-error zones (132, 186, 237) coincide with the highest-demand areas, directly identifying where proactive fleet pre-positioning is most needed.

Limitations include the single-month scope, absence of weather and event data, and heteroscedastic errors in high-traffic zones. Future work should extend the dataset to a full year, incorporate external covariates, and explore LSTM or Transformer architectures for capturing longer-range temporal dependencies.

References

References

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proc. 22nd ACM SIGKDD*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [2] Dietterich, T. G. (2000). Ensemble methods in machine learning. *Multiple Classifier Systems, LNCS*, 1857, 1–15. https://doi.org/10.1007/3-540-45014-9_1
- [3] Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in NeurIPS*, 30, 3146–3154. <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [4] NYC Taxi and Limousine Commission. (2025). TLC Trip Record Data — Yellow Taxi January 2025. NYC Open Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [6] Rodrigues, F., & Pereira, F. C. (2018). Beyond expectation: Deep joint mean and quantile regression for spatiotemporal problems. *IEEE TNNLS*, 31(12), 5377–5389. <https://doi.org/10.1109/TNNLS.2018.2855277>
- [7] Wooldridge, J. M. (2010). *Econometric Analysis of Cross Section and Panel Data* (2nd ed.). MIT Press. <https://mitpress.mit.edu/9780262232586>
- [8] Agafonov, A. (2021). Short-term traffic flow prediction using a hybrid model. *Algorithms*, 14(7), 211. <https://doi.org/10.3390/a14070211>
- [9] Gangrade, A., Pratyush, P., & Hajela, G. (2022). Taxi-demand forecasting using dynamic spatiotemporal analysis. *ETRI Journal*, 44(3), 469–480. <https://doi.org/10.4218/etrij.2021-0123>
- [10] Correa, D., & Moyano, C. (2023). Analysis and prediction of New York City taxi and Uber demands. *Journal of Applied Research and Technology*, 21(5), 886–898. https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S1665-64232023000500886